Process Design Map

ClearProcess — AI-Powered Auto Insurance Claims Management

> **Visual diagram:** see `flow_diagram.html` (open in browser → Print → Save as PDF/PNG)

System Overview

ClearProcess is a **multi-agent AI system** that orchestrates 4 specialized Claude agents across a 10-stage claims workflow. Each stage has a defined input contract, an optional AI agent, and a human approval gate before the workflow advances.

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                           CLEARPROCESS ARCHITECTURE                          │
│                                                                              │
│  STREAMLIT UI (handler + customer portal)                                    │
│         │                                                                    │
│         ▼                                                                    │
│  WORKFLOW ENGINE  ──►  STAGE HANDLER  ──►  AI AGENT (optional)               │
│         │                   │                    │                           │
│         ▼                   ▼                    ▼                           │
│  SQLITE DATABASE       CLAIM MODEL          ANTHROPIC API                    │
│  DOCUMENT STORE        (Pydantic v2)        (Claude claude-opus-4-7)         │
└──────────────────────────────────────────────────────────────────────────────┘
```

Stage-by-Stage Process Design

Stage 1 — CLAIM_REPORT (FNOL)

```
INPUT:  Claim PDF + Policy PDF  (or JSON)
        ↓ PDF text extraction (PyPDF + regex)
        ↓ Claimant, vehicle, incident data structured into Claim model
AGENT:  FNOLTriageAgent
        - System prompt: insurance coverage specialist
        - Analyzes FNOL data + policy clauses
        - Returns: decision (covered/not_covered/conditional), confidence %,
                   matched clauses, exclusions, risk flags, recommendation
        - Auto-approval if confidence ≥ 90% AND no risk flags
OUTPUT: Claim saved to DB with triage_result, fecha_aviso set
HUMAN:  Handler reviews triage result → Approve / Reject / Request documents
NEXT:   VEHICLE_INTAKE (on approval)
```

Stage 2 — VEHICLE_INTAKE

```
INPUT:  Handler clicks "Approve" in Adjuster Decision tab
        Claim.fecha_aviso must be set
AGENT:  None (NotificationAgent — disabled by default)
        Sets: fecha_ingreso_taller, workshop name, KPI entry (SLA: 240h)
OUTPUT: current_stage = VEHICLE_INTAKE
HUMAN:  Handler marks vehicle as physically received at workshop
        → sets current_stage = DAMAGE_INSPECTION, fecha_inspeccion
NEXT:   DAMAGE_INSPECTION
```

Stage 3 — DAMAGE_INSPECTION

```
INPUT:  Handler clicks "Mark vehicle received at workshop"
        vehicle_at_workshop = True, fecha_inspeccion set
        Handler uploads: inspection photos (PNG/JPG) + free-text notes
AGENT:  (Runs as part of WORK_ORDER_CREATION — see below)
        SLA: 48h maximum
OUTPUT: current_stage = DAMAGE_INSPECTION
HUMAN:  Handler uploads photos and notes → clicks "Generate Work Order"
NEXT:   WORK_ORDER_CREATION
```

Stage 4 — WORK_ORDER_CREATION

```
INPUT:  Inspection photos (bytes list) + workshop notes
        fecha_inspeccion must be set
AGENT:  WorkOrderDraftAgent
        - Multi-modal: analyzes photos + text via Claude vision
        - Tool loop: identify_damage → classify_repair_type →
                     recommend_coverage → build_work_order
        - Returns: WorkOrderLineItem[] with per-item AI recommendation
                   (cambiar / desmontar_montar / reparar_leve/mediano/grave /
                    pintar / trabajo_externo)
        - Sets: fecha_creacion_ot, work_order.phase = "draft"
OUTPUT: WO with line items, no monetary amounts yet
HUMAN:  Handler reviews each line item (SI/NO slider per item)
        → clicks "Save decisions" → phase = "handler_reviewed"
NEXT:   SPARE_PARTS_PURCHASE
```

Stage 5 — SPARE_PARTS_PURCHASE (Budget Reconciliation)

```
INPUT:  Workshop budget PDF (uploaded by handler)
        Approved WO line items (handler_approved = True items)
        fecha_creacion_ot must be set
```

```
AGENT:  WorkOrderReconciliationAgent
        - Extracts line items from budget PDF
        - Fuzzy-matches budget items to approved WO items by description
        - Fills monetary amounts: desmontar_montar, cambiar, valor_repuesto,
          reparar_leve/mediano/grave, pintar, trabajo_externo
        - Flags items in budget NOT in approved WO (is_unapproved_alert = True)
        - Sets: work_order.phase = "final"
OUTPUT: WO with all costs filled in, unapproved item warnings
HUMAN:  Handler reviews final costs + unapproved items
        Damage severity computed: Grave / Mediano / Leve
        → clicks "Start Repair" → repair_started = True
NEXT:   REPAIR_PROCESS
```

Stage 6 — REPAIR_PROCESS

```
INPUT:  Handler clicks "Start Repair" in Work Order tab
        fecha_creacion_ot must be set
AGENT:  None (notification disabled)
        Sets: repair_started = True, repair_started_at
OUTPUT: current_stage = REPAIR_PROCESS
        Customer portal shows: "Repair in progress" 🟠
HUMAN:  Workshop performs repair
        Handler clicks "Mark vehicle ready for pickup"
        → auto-chains through WORK_ORDER_CLOSURE → VEHICLE_DELIVERY
NEXT:   WORK_ORDER_CLOSURE (auto) → VEHICLE_DELIVERY
```

Stage 7 — WORK_ORDER_CLOSURE (auto-chained)

```
INPUT:  Handler clicks "Mark ready for pickup"
        fecha_creacion_ot + work_order must be set
AGENT:  None
        Sets: fecha_cierre_ot (WO formally closed)
OUTPUT: current_stage = VEHICLE_DELIVERY (auto-chained in same action)
        Customer portal shows: "Vehicle ready for pickup" 🟠
HUMAN:  — (transparent to handler, happens automatically)
NEXT:   VEHICLE_DELIVERY
```

Stage 8 — VEHICLE_DELIVERY

```
INPUT:  Handler uploads customer satisfaction receipt (PDF/image)
        fecha_cierre_ot must be set
AGENT:  None
        Sets: fecha_salida_taller
OUTPUT: current_stage = CUSTOMER_APPROVAL_BILLING
```

```
         Handler Taller tab shows: ✅ "Workshop stage complete — go to Work Order"
HUMAN:   Handler clicks "Vehicle collected" (disabled until receipt uploaded)
NEXT:    CUSTOMER_APPROVAL_BILLING
```

Stage 9 — CUSTOMER_APPROVAL_BILLING

```
INPUT:   Final invoice PDF/image (uploaded by handler)
         fecha_salida_taller must be set
AGENT:   BillComparisonAgent
         - Receives: invoice file (bytes) + approved WO line items
         - Compares invoice line-by-line against approved WO amounts
         - Returns: comparisons[] with per-line match/discrepancy,
                    total_bill_amount, total_wo_amount, total_discrepancy,
                    has_alerts, summary
         Sets: fecha_cierre, invoice_number, KPI compliance summary
OUTPUT: Claim marked COMPLETED
HUMAN:   Handler reviews AI comparison table (✅/⚠️ per line)
         Optionally adds notes → clicks "Approve invoice and close claim"
NEXT:    COMPLETED (current_stage set explicitly)
```

Stage 10 — COMPLETED

```
INPUT:   Invoice approved by handler
OUTPUT: current_stage = COMPLETED
         Customer portal shows: "Claim Completed" 🟢
         All timestamps recorded, full audit trail in DB
```

Component Interaction Diagram

```
┌──────────────┐     ┌──────────────────────────────────────────────────┐
│  HANDLER UI  │     │                     AI AGENTS                     │
│  (Streamlit) │     │                                                    │
│              │     │                                                   │
│  Triage tab  │────►│  ┌─────────────────┐  ┌───────────────────────┐  │
│  Adjuster tab│     │  │ FNOLTriageAgent │  │  WorkOrderDraftAgent  │  │
│  Taller tab  │────►│  │ Policy analysis │  │  Photo → WO items     │  │
│  WO tab      │     │  └─────────────────┘  └───────────────────────┘  │
│              │────►│  ┌──────────────────────────┐  ┌──────────────┐  │
│              │     │  │ WorkOrderReconciliation  │  │ BillCompar.  │  │
│              │     │  │ Budget fuzzy-match       │  │ Invoice check│  │
│              │────►│  └──────────────────────────┘  └──────────────┘  │
└──────────────┘     └──────────────────────────────────────────────────┘
       │                                  │
       ▼                                  ▼
┌──────────────┐             ┌──────────────────────┐
```

```
│  CUSTOMER UI │                    │   ANTHROPIC API      │
│  (Streamlit) │                    │   claude-opus-4-7    │
│              │                    │   claude-haiku-4-5   │
│  Status view │                    └──────────────────────┘
│  Doc upload  │
└──────────────┘
       │
       ▼
┌──────────────────────────────────────────────┐
│                  DATA LAYER                    │
│                                                │
│  clearprocess.db (SQLite)                      │
│  ├── claims          (full JSON blob/claim)    │
│  ├── work_orders     (derived reporting)       │
│  └── policies        (policy documents)        │
│                                                │
│  uploads/<claim_id>/                           │
│  ├── <claim_pdf>     (FNOL document)           │
│  ├── <policy_pdf>    (policy document)         │
│  ├── budget/         (workshop budget)         │
│  ├── delivery/       (conformity receipt)      │
│  ├── bills/          (final invoice)           │
│  └── customer/       (customer uploads)        │
└──────────────────────────────────────────────┘
```

Data Model — Key Entities

Claim (central entity)

```
Claim
├── id (UUID)
├── claimant (name, phone, email, policy_number)
├── vehicle (make, model, year, license_plate, VIN)
├── current_stage (WorkflowStage enum)
├── status (ACTIVE / HANDLER_APPROVED / WAITING_FOR_DOCUMENTS / REJECTED /
COMPLETED)
├── fnol_data (reporter info, incident description, photos count)
├── triage_result (decision, confidence, clauses, exclusions, risk_flags)
├── work_order (WorkOrder with line_items[], phase, costs)
├── agent_analyses[] (audit trail of all AI calls)
├── additional_documents[] (all uploaded files, base64-encoded)
├── kpi_status (SLA tracking per stage)
└── timestamps (fecha_aviso, fecha_ingreso_taller, fecha_inspeccion,
                fecha_creacion_ot, fecha_cierre_ot, fecha_salida_taller,
                fecha_cierre, repair_started_at, ready_for_pickup_at)
```

WorkOrderLineItem

```
WorkOrderLineItem
├── description
├── cambiar (float | None)          — replace labor cost
├── desmontar_montar (float | None) — disassembly/reassembly cost
├── valor_repuesto (float | None)   — parts cost
├── reparar_leve/mediano/grave (float | None)
├── pintar (float | None)
├── trabajo_externo (float | None)
├── ai_recommendation (str)
├── handler_approved (bool | None)
└── is_unapproved_alert (bool)      — added by workshop, not in original WO
```

Orchestration Logic — Workflow Engine

```
# process_stage() explicitly sets current_stage before running handler
# This allows jumping stages without sequential enforcement
workflow.process_stage(claim, WorkflowStage.WORK_ORDER_CREATION, data={
    "photo_bytes_list": [...],
    "workshop_name": "...",
    "estimated_completion_days": 10
})

# Advance loops use explicit stage sequences (not advance() which stays in place)
for target in [WORK_ORDER_CLOSURE, VEHICLE_DELIVERY, CUSTOMER_APPROVAL_BILLING]:
    if current_stage_idx >= target_idx:
        continue  # already past this stage
    result = await workflow.process_stage(claim, target, data=stage_data)
    claim = result.updated_claim
```

Security & Control Points

Control	**Implementation**
Human approval gate	Every AI output gated by explicit handler button click
Auto-approval threshold	Triage: confidence ≥ 0.90 AND zero risk flags only
Required field validation	Each stage validates prerequisites before running
Unapproved item flagging	Workshop additions highlighted with ⚠️
Document indexing	Every upload categorized and linked to requesting party
Audit trail	All AI calls logged as `AgentAnalysis` on the claim
API key management	`.env` file, never committed to source control